

1. Przygotowanie danych – Tabela płaska

Zaczynamy od stworzenia standardowej tabeli `sprzedaz_flat` i wypełnienia jej milionem testowych rekordów.

```
-- Tworzymy tabelę płaską
CREATE TABLE sprzedaz_flat (
  id SERIAL,
  data_sprzedazy DATE NOT NULL,
  kwota DECIMAL(10,2),
  klient_id INT
);

INSERT INTO sprzedaz_flat (data_sprzedazy, kwota, klient_id)
SELECT
  generate_series('2025-01-01'::date, '2025-12-31'::date, '32 seconds'::interval),
  random() * 1000,
  (random() * 10000)::int
LIMIT 1000000;

-- Dodajemy standardowy indeks na datę
```

2. Tworzenie architektury partycjonowanej

Zastosujemy **partycjonowanie RANGE** (według daty)

```
-- Tworzymy tabelę nadrzędną (logiczna struktura)

-- Tworzymy partycje fizyczne (np. kwartalne)

-- W PostgreSQL indeksy na tabeli nadrzędnej są automatycznie propagowane na partycje.

-- Przenosimy dane z tabeli płaskiej do partycjonowanej
```

4. Testy wydajności i "Partition Pruning" (15 min)

Teraz sprawdzimy, jak zachowuje się planista zapytań w obu przypadkach przy użyciu `EXPLAIN ANALYZE`.

Scenariusz: Wyciągamy dane z konkretnego miesiąca (np. lutego).

```
-- Zapytanie do tabeli PŁASKIEJ
EXPLAIN ANALYZE
```

```
SELECT * FROM sprzedaz_flat  
WHERE data_sprzedazy BETWEEN '2025-02-01' AND '2025-02-28';  
-- Zauważ: Baza przeszukuje indeks na całej tabeli miliona wierszy.
```

```
-- Zapytanie do tabeli PARTYCJONOWANEJ  
EXPLAIN ANALYZE  
SELECT * FROM sprzedaz_part  
WHERE data_sprzedazy BETWEEN '2025-02-01' AND '2025-02-28';  
  
-- Zauważ: W sekcji "Plan" zobaczysz, że baza uzyskała dostęp TYLKO do partycji  
'sprzedaz_q1'.  
-- Pozostałe partycje (q2, q3, q4) zostały całkowicie zignorowane (Pruning).
```

Zadanie :

Spróbuj zmienić datę sprzedaży jednego z wierszy tak, aby "przeskoczył" do innej partycji. PostgreSQL automatycznie wykona operację DELETE + INSERT między partycjami.